Today
you there

Hey Ron!

Welcome to your next lesson: Challenge: advanced array operations.

I’m getting your personalized lesson ready, using what you’ve learned so far and what’s ahead.

Step by step, you’re moving forward.

I am now 😊

This lesson is a challenge round on Python lists (arrays) — I’ll give you short puzzles, you solve.

Before we start:

Are you ready to jump into the first list challenge? (yes/no)

Yes, let's start

What do you think the final playlist is? Pick an option below 👇

You’re managing a playlist stored as a Python list:

```
playlist = ["Intro", "Song A", "Song B", "Song C"]

playlist.append("Song D")
playlist.insert(1, "Ad")
playlist.pop(3)
```

What is the final value of playlist?

Your answer:

A.

["Intro", "Song A", "Song B", "Song C", "Song D"]

B.

["Intro", "Ad", "Song A", "Song B", "Song C", "Song D"]

C.

["Intro", "Ad", "Song A", "Song C", "Song D"]

D.

["Intro", "Song A", "Ad", "Song B", "Song D"]

Nice work, Ron 🎯

Step by step:

Start: ["Intro", "Song A", "Song B", "Song C"]

After append("Song D"):
["Intro", "Song A", "Song B", "Song C", "Song D"]
After insert(1, "Ad"):
["Intro", "Ad", "Song A", "Song B", "Song C", "Song D"]
After pop(3) (removes item at index 3 → "Song B"):
["Intro", "Ad", "Song A", "Song C", "Song D"] ☑

You tracked the index changes correctly — that’s the key skill here 👌

Ready for challenge 2?

Yes, ready!

What do you think is the best choice here? Pick an option below 👇

You’re organizing a to-do list in Python:

todos = ["email", "meeting", "code review"]
You want to add a new task at the end: "lunch".

Which single list method is the most appropriate to use?

Your answer:

A.

todos.insert(3, "lunch")

B.

todos.append("lunch")

C.

todos[3] = "lunch"

D.

todos.remove("lunch")

Nice choice 👌

append("lunch") adds one new item exactly at the end of the list — that’s what you wanted.
insert(3, "lunch") would also work here, but it’s more fragile (depends on length); append is the clear, simple tool for “add at the end”.
todos[3] = "lunch" would try to replace an item that doesn’t exist yet.
remove("lunch") would try to delete it, not add it.

You’re picking the right tool for the job — that’s pro-level list work 👏

Ready for challenge 3 (a little bug-hunting with indexes)?

Yes, let's go!

Take your time and pick the option that best explains the bug 👇

You’re debugging a seating list for an event:

seats = ["Alice", "Bob", "Charlie", "Dana"]

seats.pop(1)
seats.insert(2, "Eli")
seats.pop(3)
A student claims the final list is:

["Alice", "Charlie", "Eli"]
What is the bug in their reasoning?

A.

They forgot that pop(1) removes "Bob", so the list length changes.

B.

They inserted "Eli" at the wrong index after the first pop.

C.

They forgot that insert(2, "Eli") puts "Eli" before the current index 2 element.

D.

They assumed pop(3) would remove "Eli", but that index no longer exists after previous changes.

a b c d
a c d
a c e d
a c e
rasoning is correct

Love that you wrote out each step, Ron 🔥

You started with:

Python

letters = ["a", "b", "c", "d"]
Then (from what you wrote):

After removing "b" → ["a", "c", "d"] ☑
After inserting "e" before "d" → ["a", "c", "e", "d"] ☑
After removing "d" again → ["a", "c", "e"] ☑

Your reasoning is solid and the final state is right 👌

Ready for challenge 4 (this time with a small 2D list / "grid")?

Python

seats = ["Alice", "Bob", "Charlie", "Dana"]

seats.pop(1)
seats.insert(2, "Eli")
seats.pop(3)
print(seats)
Nice, Ron — your code runs and the output is exactly:

Python

['Alice', 'Charlie', 'Eli']
Step by step:

Start: ["Alice", "Bob", "Charlie", "Dana"]
seats.pop(1) → remove "Bob" → ["Alice", "Charlie", "Dana"]
seats.insert(2, "Eli") → insert at index 2 → ["Alice", "Charlie", "Eli", "Dana"]
seats.pop(3) → remove index 3 ("Dana") → ["Alice", "Charlie", "Eli"] ☑

You're tracking indexes after each change really well 👏

Want to tackle the next challenge?

Yes, let's do it!

What do you think seats[0][2] is now? Pick one option below 👇

You're managing a cinema seating chart stored as a 2D Python list (each inner list is a row):

```
seats = [
    ["A1", "A2", "A3"],
    ["B1", "B2", "B3"],
    ["C1", "C2", "C3"]
]

seats[1][2] = "B3_taken"
seats[2].pop(0)
seats[0].insert(1, "A1_extra")
```

After these operations, what is the value of seats[0][2]?

Your answer:

A.

"A2"

B.

"A1_extra"

C.

"A3"

D.

"B3_taken"

Nice job on that 2D list one, Ron 🎯

Quick walk-through:

Start:

Python

```
seats = [
    ["A1", "A2", "A3"],
    ["B1", "B2", "B3"],
    ["C1", "C2", "C3"]
]
seats[1][2] = "B3_taken"
```

Only changes row 1, col 2:

["B1", "B2", "B3_taken"]

```
seats[2].pop(0)
```

Removes "C1" from the third row:

seats[2] == ["C2", "C3"]

```
seats[0].insert(1, "A1_extra")
```

First row becomes:

seats[0] == ["A1", "A1_extra", "A2", "A3"]

Now seats[0][0] = "A1", seats[0][1] = "A1_extra", seats[0][2] = "A2" ☑

You clearly see how row operations don’t affect each other’s indexes unless it’s the same inner list. Nicely done 👌

Ready for the next challenge on edge cases with lists?

Yes, I'm ready!

What do you think happens here? Take your time and choose an option 👇

You’re working with a Python list of tasks:

```
tasks = []

# Operation 1
tasks.pop()

# Operation 2
tasks.append("report")
```

What happens when this code runs?

A.

It runs fine and tasks becomes ["report"].

B.

It crashes at Operation 1 with an error about pop on an empty list.

C.

It skips Operation 1, then runs Operation 2 normally.

D.

It creates a list with one element None, then replaces it with "report".

Exactly right, Ron ✡

`tasks = []` → the list is empty.
`tasks.pop()` on an empty list raises an error: `IndexError: pop from empty list`.
Because of that error, Python stops right there, so `tasks.append("report")` never runs.

You’re watching out for empty-list edge cases — that’s a big reliability skill ☑

Last one coming up: a combo challenge with two small list tasks in one. Ready?

Let's go!

What do you think? Pick the option that matches both lists 👇

You’re doing a quick cleanup in two different Python lists.

Part 1

```
nums = [10, 20, 30, 40]
nums.pop(1)
nums.insert(2, 25)
```

What is the final value of nums?

Part 2

```
names = ["Ann", "Bob", "Cara", "Dan"]
names.remove("Bob")
names.pop()
```

What is the final value of names?

Choose the option where both list results are correct.

A.

nums = [10, 30, 25, 40] and names = ["Ann", "Bob", "Cara"]

B.

nums = [10, 30, 25, 40] and names = ["Ann", "Cara"]

C.

nums = [10, 30, 40, 25] and names = ["Ann", "Cara"]

D.

nums = [10, 20, 25, 30, 40] and names = ["Ann", "Dan"]

You nailed that combo, Ron 🔥

Step by step:

Part 1

Python

```
nums = [10, 20, 30, 40]
nums.pop(1)       # removes 20 → [10, 30, 40]
nums.insert(2, 25)  # index 2 → [10, 30, 25, 40]
So nums == [10, 30, 25, 40] ☑
```


Part 2

Python

```
names = ["Ann", "Bob", "Cara", "Dan"]
names.remove("Bob")  # → ["Ann", "Cara", "Dan"]
names.pop()          # removes last → ["Ann", "Cara"]
So names == ["Ann", "Cara"] ☑
```


You consistently tracked indexes after each change and handled both pop, insert, and remove with no confusion — that’s solid list mastery 💪

You’ve just completed the Advanced array operations challenge 🎉

You:

Correctly answered all 6 challenges
Handled multi-step mutations and index shifts
Worked with 2D lists and empty-list edge cases
Chose the right list methods for the job

You’re ready for the next lesson in Data structures & algorithms.

When you’re set, click Next Lesson in the Learn tab, and I’ll be right here to keep going with you 🚀